Lecture 14: Finite differences

Jamie Haddock

Table of contents

1 Finite differences

One of the most common applications of interpolants is numerical differentiation – approximating the derivative of a function.

> Definition: Finite-difference formula
>
> A **finite-difference** formula is a list of values $a_{-p}, \cdots, a_q$, called **weights**, such that for all f in some class of functions,
>
> $$f'(x) \approx \frac{1}{h} \sum_{k=-p}^{q} a_k f(x + kh).$$
>
> The weights are independent of f and h. The formula is said to be **convergent** if the approximation becomes equality in the limit $h \to 0$ for a suitable class of functions.

While a finite-difference formula yields a derivative estimate for a single point, typically they are applied for many points simultaneously, using a large evenly spaced grid of points.

1.1 Common examples

> Definition: Forward, backward, and centered finite-difference formulas
>
> A **forward difference formula** is a finite-difference formula,
>
> $$f'(x) \approx \frac{1}{h} \sum_{k=-p}^{q} a_k f(x + kh),$$
>
> with $p = 0$, a **backward difference formula** has $q = 0$, and a **centered difference formula** has $p = q$.

Simplest examples:

- When $p = 0$, $q = 1$, and $a_0 = -1$, $a_1 = 1$, we have *forward difference formula*
$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$
- When $p = 1$, $q = 0$, and $a_{-1} = -1, a_0 = 1$, we have *backward difference formula*
$$f'(x) \approx \frac{f(x) - f(x-h)}{h}.$$

Exercise:

Suppose $f(x) = x^2$ and $h = 1/4$ over the interval $[0, 1]$, with nodes $0, 1/4, 1/2, 3/4, 1$. Compute the forward difference estimates at $x = 0, 1/4, 1/2, 3/4$ and the backward difference estimates at $x = 1/4, 1/2, 3/4, 1$.

Answer:

We evaluate f at the nodes, $f(0) = 0, f(1/4) = 1/16, f(1/2) = 1/4, f(3/4) = 9/16, f(1) = 1$.

The forward difference estimates are:

- $f'(0) \approx 4(1/16 - 0)$, $f'(1/4) \approx 4(1/4 - 1/16)$
- $f'(1/2) \approx 4(9/16 - 1/4)$, $f'(3/4) \approx 4(1 - 9/16)$

The backward difference estimates are:

- $f'(1/4) \approx 4(1/16 - 0)$, $f'(1/2) \approx 4(1/4 - 1/16)$
- $f'(3/4) \approx 4(9/16 - 1/4)$, $f'(1) \approx 4(1 - 9/16)$

Notice that the differences are the same between the forward difference and backward difference estimates but they are interpreted as derivative estimates at different nodes.

1.2 Centered differences

The centered difference formula offers a symmetric alternative to forward and backward differences.

For simplicity, let's set $x = 0$. Note that in this case, the forward difference formula estimates $f'(0)$ as the slope of the line through $(0, f(0))$ and $(h, f(h))$, and the backward difference formula estimates $f'(0)$ as the slope of the line through $(-h, f(-h))$ and $(0, f(0))$.

That is, in these cases, we differentiate the interpolation of two points and use this as the derivative estimate.

Consider now the shortest centered formula with $p = q = 1$. Here, we have three function values (three nodes) to use, so one option is to interpolate all three points and differentiate this quadratic function,

$$Q(x) = \frac{x(x-h)}{2h^2} f(-h) - \frac{x^2 - h^2}{h^2} f(0) + \frac{x(x+h)}{2h^2} f(h).$$

This leads us to a centered difference formula

$$f'(0) \approx Q'(0) = \frac{f(h) - f(-h)}{2h}$$

which has $a_{-1} = -1/2$, $a_0 = 0$, and $a_1 = 1/2$.

In principle, we can derive any finite-difference formula in this way – interpolate nodes points on the graph of the function, then differentiate the interpolant exactly. The main motivation for increasing the number of node points involved in the finite differences formula is the improve the *order of accuracy*, which we will define soon.

```
f = x -> exp(sin(x));

h = 0.05
CD2 = (          -f(-h)   +   f(h)          ) / 2h
CD4 = (f(-2h)   -8f(-h) +   8f(h) - f(2h)) / 12h
@show (CD2, CD4);
```

```
(CD2, CD4) = (0.9999995835069508, 1.0000016631938748)
```

```
FD1 = ( -f(0)   +   f(h)        ) / h
FD2 = (-3f(0)   +  4f(h) - f(2h)) / 2h
@show (FD1,FD2);
```

```
(FD1, FD2) = (1.024983957209069, 1.0000996111012461)
```

```
BD1 = (          -f(-h) +  f(0)) / h
BD2 = ( f(-2h) - 4f(-h) + 3f(0)) / 2h
@show (BD1, BD2);
```

```
(BD1, BD2) = (0.9750152098048326, 0.9999120340342049)
```

1.3 Higher derivatives

To estimate a second (or higher) derivative, you might consider stacking finite differences of finite differences, e.g.,

$$f''(0) \approx \frac{f(-2h) - 2f(0) + f(2h)}{4h^2}.$$

However, this uses estimates at $\pm 2h$ rather than the closer values at $\pm h$. A better tactic is to return to the interpolant and compute an exact second (higher) derivative.

This yields

$$f''(0) \approx \frac{f(-h) - 2f(0) + f(h)}{h^2},$$

which is the simplest **centered second-difference formula**.

1.4 Arbitrary nodes

Equally spaced nodes are a common and convenient situation, but node locations can be arbitrary. The general form of a finite-difference formula is

$$f^{(m)}(0) \approx \sum_{k=0}^{r} c_{k,m} f(t_k).$$

The weights can be derived from the interpolate/differentiate process, but the algebra is complicated. However, there is a nice recursive algorithm that can calculate the weights for any desired formula known as **Fornberg's algorithm**.

```
  Resolving package versions...
 No Changes to `~/.julia/environments/v1.11/Project.toml`
 No Changes to `~/.julia/environments/v1.11/Manifest.toml`
```

```
"""
    fdweights(t,m)

Compute weights for the `m`th derivative of a function at zero using values at the nodes in vector `t`.
"""
function fdweights(t,m)
    # Recursion for one weight.
    function weight(t,m,r,k)
        # Inputs
        #   t: vector of nodes
        #   m: order of derivative sought
        #   r: number of nodes to use from t
```

```
        #   k: index of node whose weight is found
        if (m<0) || (m>r)        # undefined coeffs must be zero
            c = 0
        elseif (m==0) && (r==0)  # base case of one-point interpolation
            c = 1
        else
            if k<r
                c = (t[r+1]*weight(t,m,r-1,k) - m*weight(t,m-1,r-1,k))/(t[r+1]-t[k+1])
            else
                numer = r > 1 ? prod(t[r]-x for x in t[1:r-1]) : 1
                denom = r > 0 ? prod(t[r+1]-x for x in t[1:r]) : 1
                  = numer/denom
                c =  *(m*weight(t,m-1,r-1,r-1) - t[r]*weight(t,m,r-1,r-1))
            end
        end
        return c
    end
    r = length(t) - 1
    w = zeros(size(t))
    return [ weight(t,m,r,k) for k=0:r ]
end
```

fdweights

```
t = [ 0.35,0.5,0.57,0.6,0.75 ]    # nodes
f = x -> cos(x^2)
dfdx = x -> -2*x*sin(x^2)
exact_value = dfdx(0.5)
```

```
-0.24740395925452294
```

```
w = fdweights(t.-0.5,1)
```

```
5-element Vector{Float64}:
  -0.5303030303030298
 -21.61904761904763
  45.09379509379508
 -23.333333333333307
   0.38888888888888845
```

```
fd_value = dot(w,f.(t))
```

```
-0.247307422906135
```

2 Convergence of finite differences

The finite difference formulas converge to the correct value as the spacing $h \to 0$, but the number of nodes (and function evaluations) grows unbounded. We want to choose h as large as possible while still achieving acceptable accuracy.